CSCI 325 - Data Structures & Algorithms

Lab 6: Linked List Operations

Practice common linked list operations using C++ pointers and dynamic memory. Implement functions for a singly linked list, then apply the same concepts to a doubly linked list.

Two C++ starter files are provided separately on Canvas. Download both files; do not copy code from this handout. Complete the TODO functions and test the programs using the instructions below.

Activity 1: Singly Linked List

Starter file on Canvas: Lab6_Activity1_SinglyLinkedList.cpp

Download the file from Canvas, open it in your C++ editor, and complete the TODO functions. The provided insertEnd() and display() functions are already implemented. Add tests in main().

Work in the supplied Activity 1 C++ starter file. Complete the TODO functions using linked-list pointers; do not use arrays or vectors.

Tasks

1. Implement insertFront() to add a node at the beginning of the list.
2. Implement search() to return whether a specified value exists.
3. Implement countNodes() to count the nodes in the list.
4. Implement deleteValue() to delete the first occurrence of a value and release its memory.
5. Test insertion and searching. Then test deletion of the first, middle, and last nodes, and a value that does not exist. Display the list after each change.

Activity 2: Doubly Linked List

Starter file on Canvas: Lab6_Activity2_DoublyLinkedList.cpp

Download this separate file from Canvas. The provided insertEnd() and displayForward() functions are already implemented. Complete the TODO functions and add tests in main().

A doubly linked list adds a prev pointer to each node, allowing traversal in both directions. Work in the separate Activity 2 C++ starter file, and update both links when deleting nodes.

Example: nullptr <- 10 <-> 20 <-> 30 -> nullptr

Tasks

1. Implement displayBackward() by first reaching the last node and then following prev pointers.
2. Implement deleteValue(). Correctly update next, prev, and head when necessary, and release deleted memory.
3. Test deleting the first, middle, and last nodes, and a value not in the list.
4. After each deletion, display the list both forward and backward to verify the links.

Question to answer: Why does deleting a middle node in a doubly linked list require updating pointers in two directions?